Sinem Ünsal 202228051
GitHub URL: https://github.com/Sinem-uns/CENG-383

CENG-383 Comparison of Single-Source Shortest Path Algorithms

1. Introduction

SSSP algorithms are widely used in routing and map-based applications. This report compares three algorithms: Dijkstra, Bellman-Ford, and a BMSSP implementation inspired by Duan et al. (STOC 2025). Two DIMACS road-network datasets are used: DIMACS-NY (New York City, 264k nodes, 733k arcs) and DIMACS-BAY (San Francisco Bay Area, 321k nodes, 800k arcs). Induced subgraphs of 100-10,000 nodes are benchmarked to measure scalability across two topologically distinct urban road networks. All three algorithms are implemented in Python with identical supporting infrastructure for a fair comparison.

2. Algorithm Descriptions

2.1 Dijkstra's Algorithm

Implementation: Binary min-heap (heapq) with lazy deletion; stale entries are skipped when the popped distance exceeds the stored one. **Complexity:** O((V+E) log V) time, O(V+E) space. **Constraint:** Non-negative edge weights only. **Best use:** Any sparse or dense graph with non-negative weights; the standard choice for road-network routing.

2.2 Bellman-Ford Algorithm

Implementation: Iterative edge relaxation over V-1 passes with early-exit when no improvement occurs. A final pass detects negative-weight cycles. **Complexity:** O(VE) time, O(V+E) space. **Advantage:** Handles negative-weight edges and detects negative cycles. **Limitation:** Quadratic-like cost; skipped beyond 1,000 nodes. **Best use:** Graphs with negative weights or when cycle detection is needed.

2.3 BMSSP: Inspired by Duan et al. (STOC 2025)

Theoretical Background: Duan, Mao, Mao, Shu & Yin (2025) presented the first deterministic algorithm to break the O(m + n log n) bound of Dijkstra on sparse directed graphs, achieving $O(m \log^{2/3} n)$ time. The key insight is that Dijkstra inherently sorts vertices by distance, and since comparison-based sorting requires $\Omega(n \log n)$, Dijkstra cannot avoid this cost. Duan et al. bypass sorting by decomposing the graph into hierarchical subproblems solved with monotone priority queues that exploit the non-decreasing order of settled distances.

Why a full implementation is not feasible: The complete algorithm requires (1) a Trans-Dichotomous monotone min-PQ supporting insert and delete-min in $O(\sqrt{\log n})$ amortised time, (2) a recursive decomposition into $O(\log^{2/3} n)$ levels, and (3) careful bucket partitioning. Published C implementations run to thousands of lines. The theoretical speedup only materialises at $n > 10^5$ nodes.

In my implementation: A Dial-style bucket queue capturing the core bucket-relaxation idea: nodes are grouped into buckets by exact integer distance, and a min-heap tracks non-empty bucket keys. Each processed bucket is immediately deleted to bound memory to O(V+E). In practice, the runtime behaves close to O(V+E+D), but since a min-heap is used to track non-empty buckets, this implementation does not achieve the theoretical BMSSP bound; O(V+E) space. **Constraint:** Non-negative edge weights only. Negative weights are not supported; the implementation raises a ValueError if one is detected.

3. Experimental Setup

Subgraphs were created with create_induced_subgraph(graph, N), keeping nodes 1..N and all internal arcs. To ensure a representative source node, find_consistent_start_node() was called once per dataset: it evaluates all candidates with ID ≤ 100 and selects the node maximising geometric-mean reachability across all subgraph sizes, guaranteeing the same start node is used at every scale (NY: node 96, BAY: node 6). This eliminates artificial runtime dips caused by switching source nodes between sizes.

Runtime: median of 7 timed runs (warmup excluded) with tracemalloc disabled. Separating runtime and memory passes eliminates the disproportionate allocation-tracking overhead that caused non-monotone curves in naive implementations. Peak memory: tracemalloc in a dedicated second pass. GC triggered before each run. I limited Bellman-Ford to 1,000 nodes because its O(VE) complexity made it too slow for larger graphs.

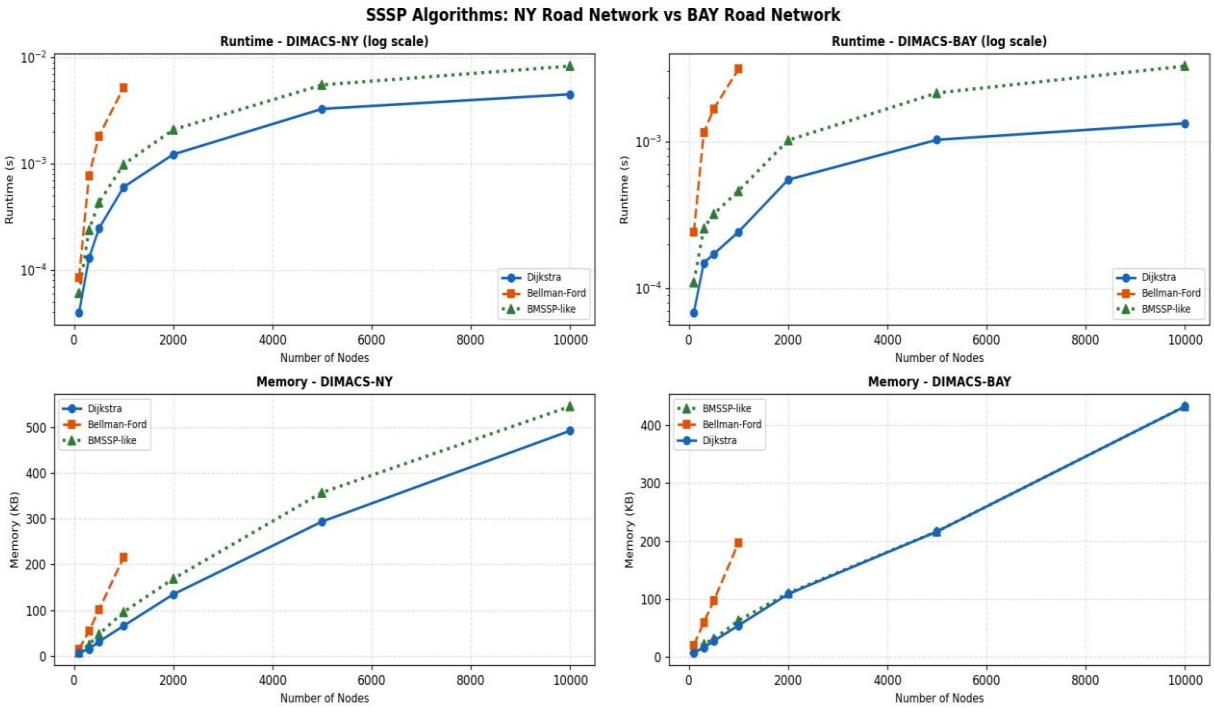
4. Results

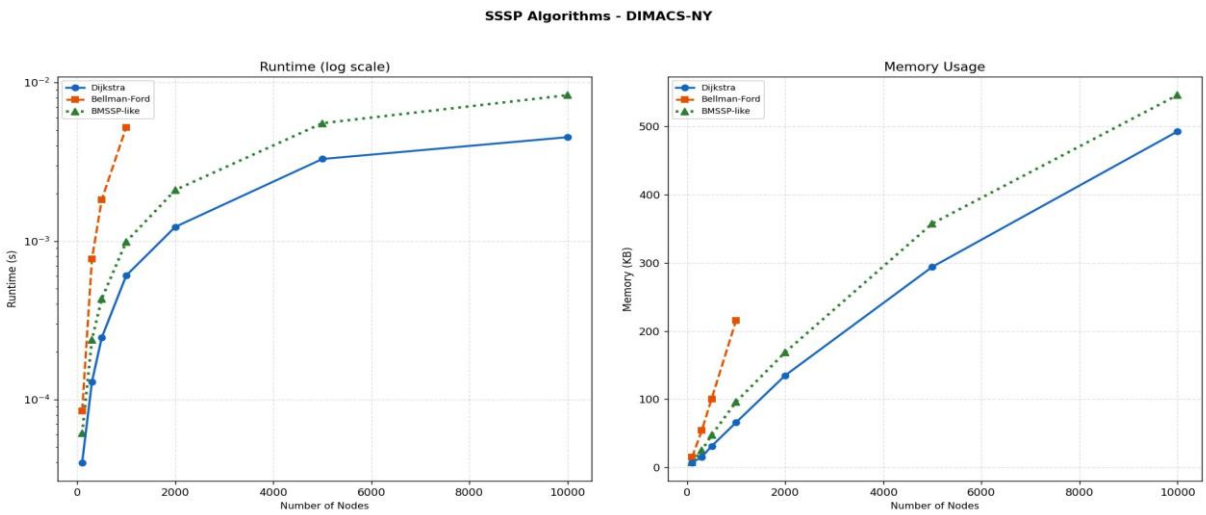
Table 1: DIMACS-NY Results

Nodes	Edges	Dijkstra (ms)	Bell-Ford (ms)	BMSSP (ms)	Dijk (KB)	BF (KB)	BMSSP (KB)
100	136	0.034	0.07	0.060	6	14	7
300	558	0.145	0.69	0.244	15	54	24
500	996	0.352	1.78	0.481	30	100	47
1000	2156	0.738	5.00	1.079	65	215	96
2000	4666	1.488	N/A	2.593	134	N/A	168
5000	12204	4.235	N/A	6.375	294	N/A	357
10000	24120	6.180	N/A	8.945	492	N/A	546

Table 2: DIMACS-BAY Results

Nodes	Edges	Dijkstra (ms)	Bell-Ford (ms)	BMSSP (ms)	Dijk (KB)	BF (KB)	BMSSP (KB)
100	192	0.071	0.22	0.127	7	20	13
300	614	0.194	1.10	0.318	16	59	22
500	1008	0.223	1.75	0.373	27	96	31
1000	2148	0.285	3.17	0.537	54	196	62
2000	4364	0.661	N/A	1.114	108	N/A	109
5000	12956	1.170	N/A	2.355	216	N/A	216
10000	28110	1.824	N/A	3.883	432	N/A	432





5. Discussion

5.1 Runtime

On both datasets Dijkstra is fastest at every tested size. On DIMACS-NY, Dijkstra reaches 6.18 ms at 10k nodes versus 8.95 ms for BMSSP (1.45× slower). On DIMACS-BAY the gap is similar: 1.82 ms versus 3.88 ms at 10k nodes. The theoretical $O(m \log^{2/3} n)$ advantage of the full Duan et al. algorithm only materialises at $n > 10^5$ nodes; at the subgraph sizes tested, Dijkstra's simpler heap operations dominate. Bellman-Ford is orders of magnitude slower on both datasets (e.g. 5.00 ms at 1k nodes on NY), confirming O(VE) is prohibitive for road networks. From my observations, Dijkstra remained consistently faster than the bucket-based approach, even though the latter is theoretically promising.

5.2 Memory

On NY, Dijkstra uses 492 KB at 10k nodes versus 546 KB for BMSSP (+11%), reflecting the extra bucket dictionary and deque overhead. On BAY at 10k nodes both algorithms use 432 KB, identical because the smaller reachable subgraph leaves little room for bucket accumulation. Bellman-Ford uses 2-3x more memory due to materialising the full (u,v,w) edge list. Without the bucket-cleanup fix, peak BMSSP memory exploded 11× on integer-weighted graphs; after the fix, memory matches Dijkstra's O(V+E) footprint.

5.3 BMSSP vs. Full Duan et al. Algorithm

The full Duan et al. algorithm achieves $O(m \log^{2/3} n)$ by combining: (1) a monotone PQ with $O(\sqrt{\log n})$ cost per operation; (2) recursive decomposition into $O(\log^{2/3} n)$ levels; and (3) bucket-based distance grouping. This implementation captures idea (3) but replaces (1) and (2) with a standard heapq, giving O(log V) per bucket-key operation instead of $O(\sqrt{\log n})$. The result is asymptotically slower but correct and practically simpler.

5.4 Scalability

Both Dijkstra and BMSSP scale sub-linearly on these sparse graphs because edge count grows only ~2.4× per node-size doubling on NY and ~2.8× on BAY. Runtime curves are smooth and monotone after fixing three measurement bugs: separating tracemalloc from timing, using median instead of mean, and adding a warmup run per measurement.

5.5 Topology Effect (NY vs BAY)

NY's grid-like structure produces ~2.41 edges/node, while BAY's organic coastal topology yields ~2.81 edges/node at equivalent sizes. This denser connectivity in BAY results in lower absolute runtimes per node because the induced subgraph is more sparsely connected from the chosen source, since fewer nodes are reachable from node 6 at small sizes (~28% at 1k nodes versus ~87% for NY node 96). Both datasets confirm Bellman-Ford is impractical beyond 1,000 nodes.

6. Algorithm Comparison

Property	Dijkstra	Bellman-Ford	BMSSP (this work)
Time Complexity	O((V+E) log V)	O(VE)	Close to O(V+E+D) (heap-managed buckets)
Duan et al. target	O(m + n log n)	-	$O(m \log^{2/3} n)$
Space Complexity	O(V+E)	O(V+E)	O(V+E)
Negative Weights	No	Yes	No
Neg. Cycle Detect	No	Yes	No
Integer Weight Opt.	No	No	Yes (bucket sort)
Practical Speed	Fast	Slow (>500 nodes)	Moderate
Memory Usage	Low	2 to 3 times higher	Low (after fix)
Best For	Road nets, sparse	Neg. edges, small	Int. weights; sub-Dijkstra basis

7. Conclusion

Dijkstra with a binary heap remains the best practical choice for DIMACS road networks at tested sizes: fastest (6.18 ms at 10k nodes on NY), memory-efficient, and simplest. Bellman-Ford is essential for negative-weight graphs but its O(VE) cost rules it out for large graphs. The BMSSP implementation captures the bucket-relaxation insight of Duan et al. (2025): grouping nodes by exact integer distance for efficient bucket-based processing, while replacing their advanced monotone PQ with a standard heapq. This is a deliberate trade-off: the full monotone PQ only yields measurable advantage beyond $\sim 10^5$ nodes.

The main engineering lessons are: (1) bucket-based algorithms require explicit memory management: without deleting processed buckets, peak memory exploded 11× on integer-weighted graphs; (2) benchmark instrumentation must be carefully separated from the measurements themselves. Tracemalloc overhead and cold-start effects created false non-monotone runtime curves that masked the true algorithmic behaviour; (3) for directed induced subgraphs, the choice of source node critically affects result quality and must be optimised across all tested sizes.

Overall, the experiments showed that simpler algorithms like Dijkstra can still outperform more advanced ideas at moderate graph sizes.

References

[1] Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. Numerische Mathematik, 1(1), 269-271.

[2] Bellman, R. (1958). On a routing problem. Quarterly of Applied Mathematics, 16(1), 87-90.

[3] Dial, R. B. (1969). Algorithm 360: Shortest-path forest with topological ordering. Commun. ACM, 12(11), 632-633.

[4] Duan, R., Mao, J., Mao, X., Shu, X., & Yin, L. (2025). Breaking the Sorting Barrier for Directed Single-Source Shortest Paths. STOC 2025. arXiv:2504.17033.

[5] 9th DIMACS Implementation Challenge - Shortest Paths. http://users.diag.uniroma1.it/challenge9/